

Sparse Carried-State TTT for V-JEPA 2.1

Burn JEPA Contributors

Preprint draft, May 2026

Abstract

V-JEPA 2.1 produces dense video features with a tubelet encoder, but dense patchification and dense token processing are a poor fit for streaming systems whose visible regions are already sparse. We present a Burn-native sparse V-JEPA 2.1 implementation and an encoder-only training recipe, Sparse Carried-State TTT (SC-TTT), that carries recurrent fast weights across adjacent video windows while retaining deployable free-run inference semantics. This is an empirical systems contribution, not a claim of theoretical originality or state-of-the-art video representation learning. The implementation maps all 312 tensors from the official `vjepa2.1_vit_base_384` checkpoint and matches the upstream `torch.hub` sparse encoder/predictor to 2.92×10^{-5} max absolute error. A verified SC-TTT checkpoint trained with packed-stream TBPTT, sparse context rollout, dense stabilization samples, state/update regularization, Memory-ALiBi carried state, and a small norm-only continuation closes most of the sparse image-token gap to a dense image baseline when distilling to the 3D video teacher: on a matched slice, base sparse image tokens score 0.4631/0.8048 loss/cosine, dense image tokens score 0.3288/0.8636, and sparse SC-TTT scores 0.3410/0.8576. The current stage2 checkpoint remains stable in explicit long-run stress gates: 0.2794/0.8851 over a 1088-window same-stream repeat with late-early loss -0.0009 , and 0.2573/0.8924 over a 512-window adversarial stitch with late-early loss -0.0193 . We separately implement an In-Place-TTT-inspired MLP path and a stricter conformance lane, but the shipped result remains the SC-TTT adapter/Memory-ALiBi model because the strict lane is currently only smoke-validated and not quality-promoted. The main remaining gates are larger open-set video, repeated measurements, downstream baselines, and lower-cost backward/optimizer kernels.

1 Introduction

V-JEPA 2.1 is a joint-embedding predictive architecture for image and video self-supervision [1]. Its video path is naturally dense: for a 16-frame 256px clip with 16px patches and tubelet size 2, the encoder sees 2048 video tokens before masking. Streaming perception pipelines often know that only a subset of image regions should be refreshed, for example from AutoGaze or patch-difference attention. The question in this paper is whether V-JEPA 2.1 can use those sparse regions directly while retaining some of the temporal information normally supplied by dense video tubelets.

We study this as a systems and distillation problem. The contribution is a measured implementation and training protocol: Burn-native V-JEPA 2.1 parity, sparse positional indexing, sparse GPU patchification, sparse encoder-side TTT rollout, stream-aware TBPTT training, and diagnostics that separate deployable free-run behavior from teacher-forced or dense diagnostic paths. We do not claim that the method is theoretically original, the first sparse V-JEPA system, or state of the art. The current evidence is strong enough to support a reproducible engineering method and a research agenda; it is not yet a final production model.

Recent LeJEPA and LeWorldModel work argues that JEPA training can be stabilized by adding an explicit Gaussian latent regularizer instead of relying only on EMA, stop-gradient, architectural priors, or multi-term loss stacks [3, 4]. Weak-SIGReg further motivates covariance-sketch variants as a cheaper regularization surface [5]. We adopt this direction conservatively: the teacher remains the official pretrained V-JEPA 2.1 model, and the added term is a small student-token stabilizer rather than a replacement for teacher distillation.

2 Problem Statement and Scope

Problem statement. Let $W_i \in \mathbb{R}^{B \times C \times T \times H \times W}$ be the current rolling teacher window from a stream, with $T = 16$ frames in the reported runs, and let G be the V-JEPA tubelet grid. The window contains only the current teacher target horizon; earlier context must reach the student through carried state rather than by extending the teacher sample with future or privileged frames. A sparse context mask $M_i \subseteq G$ selects the tokens refreshed for that window. Given a frozen dense V-JEPA teacher E_θ , learn an encoder-only student S_ψ with recurrent TTT state F_i such that

$$S_\psi(W_i, M_i, F_{i-1}) \approx E_\theta(W_i) \quad (1)$$

on the selected evaluation tokens, while avoiding dense patchification and while updating F_i without privileged teacher tokens at inference time.

Assumptions. The experiments and system design rely on the following assumptions.

1. The loaded checkpoint is the official V-JEPA 2.1 ViT-B checkpoint and sparse encoder/predictor parity is checked before quality claims.
2. Sparse masks are available before JEPA patchification. They may come from AutoGaze, patch-difference logic, or precomputed manifests.
3. The deployed student updates TTT state from detached student hidden states. Teacher-forced updates are diagnostics only.
4. Teacher supervision is a rolling 16-frame V-JEPA video target in the production configs. Longer temporal context is supplied by carried TTT state, not by giving the teacher future frames inside one target sample.
5. Long-running streams use the trained state dynamics and diagnostic reset/decay policies. Simple unbounded carried state was a measured failure in earlier stress gates; the current Memory-ALiBi continuation is evaluated with much longer same-stream and adversarial-stitch gates, but still should not be read as a broad arbitrary-distribution stability guarantee.
6. Feature distillation against the dense teacher is a proxy objective. We do not yet report downstream task accuracy.
7. Frozen sparse-patchify training is the intended adapter-training bridge. Full unfrozen finetuning is a higher-risk ablation.

When these assumptions are violated, the expected failure mode changes. If the mask misses semantic updates, the feature cache can become stale. If state is carried with dynamics that were not trained or measured, fast weights can decohere. If scene switches are stitched without state dynamics or reset coverage that matches training, the state can contaminate the next scene. If dense diagnostic probes are enabled over large full-token tensors, the current CUDA backend can exhaust allocator pages.

3 Definitions and Formal Guardrail

This section is included to make the evaluation terminology precise. It is not a claim that the paper is theoretically original.

Definition 1: Carried-state coherence gap. For an ordered stream dataset $\mathcal{D} = \{(V_i, M_i)\}_{i=1}^n$, let L_{run} and C_{run} be the average feature loss and cosine similarity of the deployable free-run student with persistent state $F_i = U_\psi(F_{i-1}, V_i, M_i)$. For a diagnostic intervention q such as reset-each-frame, freeze-fast-update, shuffle, or reverse order, define

$$\Delta_L(q) = L_q - L_{\text{run}}, \quad (2)$$

$$\Delta_C(q) = C_{\text{run}} - C_q. \quad (3)$$

Positive $\Delta_L(q)$ and $\Delta_C(q)$ mean that the persistent temporal state is useful relative to intervention q . Longitudinal drift is measured by $L_{\text{late}} - L_{\text{early}}$ and $C_{\text{late}} - C_{\text{early}}$ over ordered eval-window segments. This refines an ordinary average eval loss by testing whether memory helps and whether it remains stable through time.

Proposition 1: zero-output TTT insertion is initially function-preserving. Assume a pretrained encoder block computes $X \mapsto B_\theta(X)$, and a TTT adapter is inserted as a residual

$$A_\psi(X, F) = X + W_o \phi_\psi(X, F), \quad (4)$$

where $W_o = 0$ at initialization. Then the augmented block output equals the pretrained block output for every input X and fast state F before training.

Proof. At initialization, $W_o = 0$. Therefore

$$A_\psi(X, F) = X + 0 \cdot \phi_\psi(X, F) = X.$$

Replacing the identity residual path before or after the pretrained block therefore does not change the value passed to the rest of the encoder. By composition over all inserted adapters, the augmented encoder is identical to the pretrained encoder before adapter training. The argument assumes exact zero initialization of the adapter output projection and identical positional and token-order inputs; it does not prove anything about trained fast-state stability. $\square$

4 Method: Sparse Carried-State TTT

4.1 Sparse V-JEPA Forward

For context mask M_c and target mask M_t , sparse V-JEPA computes

$$z_c = E_\theta(V, M_c), \quad \hat{z}_t = P_\theta(z_c, M_c, M_t). \quad (5)$$

Sparse plans cache token indices, sequence order, coordinates, and positional encoding tensors. The CUDA and WGPU sparse-patchify paths use `burn_flex_gmm` so masked pixels are skipped before patch embedding instead of gathered after dense patchification.

4.2 SC-TTT Adapter

Each selected encoder layer receives a zero-output residual adapter with a fast state F . For hidden tokens X , the adapter reads memory with

$$X + W_o(XF/\sqrt{d}), \quad (6)$$

then updates F from chunked, detached student hidden states. The output projection is zero-initialized, so the untrained adapter starts as a no-op by Proposition 1. We place TTT modules only in the encoder because the shipped artifact is an encoder representation; predictor-side adapters would adapt a training head and add work after sparse token extraction.

4.3 In-Place TTT Conformance Lane

In-Place TTT [7] motivates a different integration point: selected pretrained MLP down-projections become the base fast weights rather than adding standalone residual adapters. We implement this as an ablation, not as the default SC-TTT model. The non-strict `in_place_mlp` lane reuses selected V-JEPA MLP down-projections but keeps older local update semantics. The stricter `in_place_mlp_strict` lane is the conformance-oriented path: it uses the selected MLP `fc2` as the fast-weight base, uses causal target generation, applies the current fast weight before updating it, and updates only on full chunks. It intentionally does not use Memory-ALiBi, because multi-bank state is a local SC-TTT extension rather than the single fast-weight cache used by the In-Place TTT formulation.

The paper’s empirical claims therefore stay separated. Tables and deployment results labeled SC-TTT use the residual adapter/Memory-ALiBi path. The strict in-place lane is currently a conformance and performance ablation pending a long matched training run and reference-step parity fixture; it is not the promoted checkpoint.

4.4 Latent Stability Regularizer

The selected run adds a low-weight LeJEPa/SIGReg-inspired penalty to the student tokens used by the feature loss. For aligned student tokens $Z \in \mathbb{R}^{N \times d}$, the implemented regularizer is

$$R(Z) = \lambda [\alpha_\mu \|\mu(Z)\|_2^2 + \alpha_\sigma \|\text{var}(Z) - \mathbf{1}\|_2^2 + \alpha_c C_{\text{sketch}}(Z)], \quad (7)$$

where C_{sketch} is a deterministic adjacent-feature covariance sketch over the first 64 feature dimensions. The implementation is device-resident and mask-aware; padded ragged tokens are excluded from the mean/variance estimates. This is not a full LeJEPa replacement objective: the V-JEPa 2.1 teacher remains fixed, and the regularizer is used only as a small anti-collapse/stability term for the student rollout.

Algorithm 1: Sparse Carried-State TTT (SC-TTT).

1. Load the official V-JEPa 2.1 teacher and initialize an encoder student with zero-output TTT adapters at layers [3, 7, 11].
2. For each packed stream batch, restore one fast state per stream key.
3. Resolve sparse context/target masks from the manifest or mask driver.
4. Run the frozen dense teacher on the current rolling 16-frame video window and detach the teacher features.
5. Roll the student through sparse frame/tubelet inputs, updating fast state from detached student hidden states.
6. Minimize feature MSE between student tokens and selected teacher tokens, optionally plus $R(Z)$.
7. Detach the carried fast state at the TBPTT boundary, apply decay, and store it under the stream key.
8. Periodically include dense stabilization samples and scheduled resets so the adapter learns both zero-state initialization and carried-state use.

5 Experimental Protocol

All numerical correctness claims use the official `vjepa2.1.vit_base_384` checkpoint loaded from Meta’s V-JEPa 2.1 `torch.hub` entry point. Quality runs use local open-set video windows at 256px with 16-frame student clips and a frozen 3D teacher. The production configs set `dataset.frames=16` and `training.teacher_window_frames=16`; this makes each teacher target a rolling current-window target rather than an expanding or future-looking teacher sample. The verified SC-TTT run uses sparse context masks with 410 of 2048 tokens (20.0%) and 103 target tokens (5.0%).

Long-rollout eval uses sequential manifest order with batch size 1 so state is actually carried across adjacent windows. Packed-stream eval is useful for throughput, but round-robin stream packing can hide whether a single stream is being carried continuously. Diagnostic probes compare the deployable free-run state against reset-each-frame, reset-each-tubelet, shuffle, reverse, and freeze-fast-update interventions.

Image-to-video controls are required for fair interpretation. A sparse non-TTT baseline must be trained or measured against the same 3D teacher target; otherwise the comparison silently assumes that pretrained image-token features and video-tubelet features already live in the same distribution. We therefore report dense image, sparse image, SC-TTT, no-TTT norm-only, no-TTT last-block, and TTT continuation probes separately where available.

Backend labels are conservative. Burn exposes both `webgpu` and `wgpu` feature gates; they are related WebGPU-family lanes unless a row specifically measures the native WGPU `burn.flex.gmm` extension.

Table 1: V-JEPA 2.1 ViT-B parity against the official `torch.hub` module. Entries are max absolute error.

Case	Sparse encoder	Predictor	Target tokens
Micro sparse 2x2	1.109×10^{-5}	1.812×10^{-5}	1.034×10^{-5}
Multi-tubelet 3x4	1.335×10^{-5}	1.597×10^{-5}	2.921×10^{-5}

Table 2: Verified checkpoints and continuations. The current active candidate is the norm-only stage2 continuation from the Memory-ALiBi checkpoint; the earlier SIGReg and long-stable rows are retained as historical ablations.

Run	Steps	Samples	Best train loss	Final sampled loss
SC-TTT + SIGReg selected	160	320	0.1829	0.3111
SC-TTT long-stable continuation	128	256	0.1653	0.2479
SC-TTT Memory-ALiBi + norms	512	1024	0.1649	0.2549
SC-TTT unregularized	160	320	0.1828	0.3111
Low-LR continuation	96	192	0.2555	0.2555

6 Results

6.1 Numerical Correctness

The loader applies all 312 mapped tensors with no missing, skipped, or errored tensors. The multi-tubelet case exercises temporal indexing, non-square spatial position handling, sparse context gather, target gather, and predictor ordering.

6.2 Verified Checkpoint and Long-Rollout Coherence

The SIGReg run exercises the intended training path: 211 carried windows, 109 reset windows, 320 detached/decayed states, and dense stabilization samples on 55 of 160 optimizer steps. The long-stable continuation starts from SIGReg, keeps the same encoder-only adapter placement, trains on repeated continuous streams, and ends with a reset interval of 16 windows. It contributes 184 carried windows, 72 reset windows, and 256 detached/decayed states. The low-LR continuation improved sampled training loss but is not selected because its long-rollout eval worsened. The stage2-norms continuation starts from the Memory-ALiBi candidate, keeps sparse patchify frozen, adds only 43,008 pretrained LayerNorm trainable parameters, and is the current promoted checkpoint in this draft.

Tables 7–10 are the current long-rollout gate. The selected checkpoint is coherent under the short diagnostic gate: persistent memory is better than resetting, freezing, shuffling, or reversing temporal order. The full-manifest gate adds the direct V-JEPA 2.1+sparse versus V-JEPA 2.1+SC-TTT+sparse ablation. SC-TTT improves over base sparse by 0.1064 loss and 0.0487 cosine, and over reset-each-window by 0.0781 loss and 0.0366 cosine. The SIGReg-style regularizer is a safe hook but does not materially change the full-manifest result: feature loss shifts from 0.3208981 to 0.3208972.

The stress gate is deliberately less favorable. It repeats one stream beyond the observed held-out horizon, stitches unrelated streams into one artificial state trajectory, and alternates scenes adversarially. The result is the central limitation and the central fix: simple unbounded carried state fails, while explicit state dynamics and reset policies matter. The later Memory-ALiBi + norms continuation improves both the 1088-window same-stream gate and the 512-window adversarial-stitch gate, so the draft now treats stage2-norms as the active candidate. The claim is still empirical and scoped: these gates exercise longer carried state than the manifest, but they do not prove arbitrary-distribution stability.

6.3 Mask Policies

This table does not justify a universal AutoGaze-is-best claim. At matched density, AutoGaze-style and patch-diff masks are close on teacher-feature quality. AutoGaze’s practical advantage in the current pipeline

Table 3: LeJEPA/SIGReg-style latent regularizer ablation. The 64-step rows use a matched 24-window held-out subset; the selected row is the full 160-step checkpoint evaluated on the full 164-window sequential manifest. The added term is non-regressive but effectively neutral at this scale.

Run	Reg weight	Eval samples	Feature loss	Cosine	Reg loss
64-step off	0	24	0.3478696	0.8516861	–
64-step low	1e-5	24	0.3478668	0.8516874	5.82e-6
64-step high	5e-5	24	0.3478670	0.8516872	2.91e-5
160-step selected	1e-5	164	0.3208972	0.8674483	6.19e-6

Table 4: Image-student to video-teacher alignment controls. The sparse SC-TTT row uses the deployed recurrent path; dense and sparse image rows are base V-JEPA 2.1 image-token baselines against the same rolling 3D teacher target. Lower loss and higher cosine are better.

Lane	Student tokens	Loss	Cosine
Dense image V-JEPA 2.1 baseline	2048 / 2048	0.3288	0.8636
Sparse image V-JEPA 2.1 baseline	410 / 2048	0.4631	0.8048
Sparse Memory-ALiBi SC-TTT	410 / 2048	0.3410	0.8576

is that precomputed masks avoid patch-diff scoring and selection overhead.

6.4 Throughput

The CUDA sparse-patchify lane is the intended inference path. WebGPU-family throughput can be lower than CPU ndarray in low-density E2E rows because the AutoGaze/mask side performs many short launches and synchronization points; it is not evidence that dense CPU JEPA is faster than device-resident sparse GPU JEPA.

Training speedups are not proportional to token reduction. Adapter backward, AdamW, transformer dispatch, and fixed kernel overhead remain in the timed loop. In the selected checkpoint run, backward plus optimizer consumed 414.7 s of 489.7 s train elapsed.

7 Failure Cases and Limitations

The evaluation includes cases where the method loses or the system fails. First, the low-LR continuation improves sampled train loss but worsens sparse carried eval from 0.3334/0.8629 to 0.3977/0.8410, so train loss alone is not a sufficient model-selection criterion. Second, the LeJEPA/SIGReg-style regularizer is non-regressive but only neutral on the current V-JEPA 2.1 distillation run; it should be treated as a stability hook, not as evidence that Gaussian regularization solves long-rollout drift here. Third, simple unbounded TTT state carry was a measured failure before the Memory-ALiBi continuation: the repeated same-stream gate drifted by +0.1772 loss over 136 windows, and the stitched-scene gate drifted by +0.2124 loss without scene reset. Fourth, dense full-grid temporal diagnostics over large token tensors currently hit a CubeCL CUDA allocator panic; dense primary eval works, but dense diagnostic probes need chunking or streaming. Fifth, the 164-window held-out manifest check improves with persistent state on average but has only 33 contiguous windows in one stream, so it was insufficient by itself. Sixth, the strict in-place MLP lane is not promoted: it passes causal/update smoke tests and a real-checkpoint CUDA smoke, but it has not yet beaten the adapter baseline or passed reference-step parity against the upstream implementation. Seventh, patch-diff masks can miss slow semantic changes; refresh policies such as age-priority, blue-noise refresh, or keyframes are required for persistent feature caches.

The largest current quality run is still small compared with open-set video pretraining. The paper reports teacher-feature distillation metrics rather than downstream task accuracy. Results are mostly single-run measurements; larger repeated sweeps are needed before claiming robustness or generality.

Table 5: Stage2 trainable-capacity probes. No-TTT controls train the image pathway against the same video-teacher feature loss, so TTT is not compared only against an unfine-tuned image baseline.

Lane	Steps	Dense/sparse steps	Trainable params	Pre loss/cos	Post loss/cos
No-TTT image-to-video, norms	96	65 / 31	43K	0.3981 / 0.8293	0.3954 / 0.8304
No-TTT image-to-video, last 2 blocks	96	65 / 31	14.18M	0.3981 / 0.8293	0.3967 / 0.8299
TTT + norms, from Memory-ALiBi best	96	65 / 31	3.59M	0.3557 / 0.8477	0.3493 / 0.8504
TTT + norms, from Memory-ALiBi best	192	66 / 126	3.59M	0.3557 / 0.8477	0.3465 / 0.8516

Table 6: Bounded real-checkpoint CUDA conformance smoke for the In-Place TTT lanes. Both rows use rolling 16-frame teacher windows and deployable self-hidden memory updates. This is a four-step initialization and plumbing check, not a promotion-quality training result.

Lane	Insertion	Eval loss	Eval cosine	Samples/s
Current SC-TTT adapter baseline	adapter	0.4136	0.8261	0.0759
Strict in-place MLP conformance	in_place_mlp_strict	0.4534	0.8091	0.0854

8 Research Claim Audit

Table 15 applies the research contribution checklist directly. The correct classification is a measured systems method with an algorithmic recipe. It should not be described as theoretically original. It should also avoid the phrase “novel algorithm” until broader external baselines, repeated statistics, and downstream task results are available.

Final gate answers. The new concept is the carried-state coherence gap for deployable sparse TTT rollouts. The formal claim is the zero-output adapter insertion proposition. The proof is algebraic and only covers initialization. The algorithmic recipe is SC-TTT. It beats base sparse V-JEPA 2.1 on the full 164-window manifest, beats reset/freeze/order-intervention baselines on the short verified sparse long-rollout eval, and beats reset-each-window on the 164-window held-out manifest. The stage2 Memory-ALiBi + norms continuation improves the long same-stream and adversarial-stitch gates further. It fails when state dynamics do not match the deployment stream, when train-loss-only continuation worsens carried eval, when the strict in-place MLP lane is used before it beats the adapter baseline, when the latent Gaussian regularizer is only neutral, when diagnostic dense probes exceed backend memory behavior, and when masks miss semantic updates. Those failures violate the assumptions that eval selection matches deployment, diagnostics are chunked, sparse masks cover semantic changes, and the measured state dynamics are active. Future work should scale data, add external and downstream baselines, repeat measurements, add an exact In-Place TTT reference fixture, chunk dense diagnostics, and improve sparse TTT backward/optimizer kernels.

9 Reproducibility Notes

Selected artifact:

```
target/burn-jepa-production-final-256/
stage1-stream-tbptt-sigreg/ttt-model.mpk
stage1-stream-tbptt-longstable/ttt-model.mpk
stage2-norms-low-lr/ttt-model.mpk
```

Training command:

```
cargo run --no-default-features --features cuda,sparse-patchify-cuda \
--bin burn-jepa -- train-ttt \
--config configs/production/vjepa21-ttt-stage1-stream-tbptt-sigreg-cuda.toml

cargo run --no-default-features --features cuda,sparse-patchify-cuda \
--bin burn-jepa -- train-ttt \
--config configs/production/vjepa21-ttt-stage1-stream-tbptt-longstable-cuda.toml
```

Table 7: Short sequential carried-state eval. These rows use free-run student state, not teacher-forced adapter updates.

Eval	Samples	Carried/reset	Loss	Cosine
Sparse SC-TTT selected	24	21 / 3	0.3334	0.8629
Sparse low-LR continuation	24	21 / 3	0.3977	0.8410
Dense selected	16	15 / 1	0.3946	0.8448

Table 8: Full held-out manifest sparse ablation. The base sparse row is V-JEPA 2.1 with the same sparse context masks and zero-output TTT insertion, which is function-preserving before adapter training. The manifest has 164 windows across 19 streams; the longest contiguous stream is 33 windows.

Eval	Samples	Carry/reset	Longest	Loss	Cosine	Late-early loss
V-JEPA 2.1 sparse	164	145 / 19	33	0.4273466	0.8187255	+0.0405
SC-TTT + SIGReg sparse	164	145 / 19	33	0.3208972	0.8674483	+0.0838
SC-TTT sparse unregularized	164	145 / 19	33	0.3208981	0.8674482	+0.0838
SC-TTT reset each window	164	0 / 164	33	0.3989558	0.8308782	+0.0417

```
cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- train-ttt \
  --config configs/production/vjepa21-ttt-stage2-norms-low-lr-cuda.toml
```

Sparse long-rollout eval:

```
MODEL=target/burn-jepa-production-final-256/stage1-stream-tbptt-sigreg/ttt-model.mpk
cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-ttt-long-rollout-sigreg-cuda.toml \
  --model "$MODEL" \
  --steps 164 --batch-size 1 --no-full-grid
```

Full held-out manifest sparse ablation:

```
MODEL=target/burn-jepa-production-final-256/stage1-stream-tbptt-sigreg/ttt-model.mpk
cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-base-sparse-long-rollout-verylong-cuda.toml \
  --base-sparse \
  --steps 164
```

```
cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-ttt-long-rollout-sigreg-cuda.toml \
  --model "$MODEL" \
  --steps 164
```

```
cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-ttt-long-rollout-reset-window-cuda.toml \
  --model "$MODEL" \
  --steps 164
```

Long-rollout stress gates:

```
LONG=target/burn-jepa-production-final-256/stage2-norms-low-lr/ttt-model.mpk
```

Table 9: Long-rollout stress gates added after the full manifest revealed that 33 contiguous windows were insufficient evidence. No-reset rows are diagnostic failures of unbounded carried state; reset-16 and scene-reset rows use the long-stable continuation. Lower late-early loss is better.

Gate	Windows	Switches	Resets	Loss/cosine	Late-early loss
Same stream, no reset	136	0	1	0.4982 / 0.8046	+0.1772
Same stream, reset32	68	0	3	0.3720 / 0.8510	+0.0837
Same stream, reset16	136	0	9	0.3270 / 0.8666	+0.0049
Scene stitch, no scene reset	64	10	1	0.4468 / 0.8218	+0.2124
Scene stitch, scene reset	64	10	11	0.2896 / 0.8783	-0.0675
Adversarial stitch, scene reset	64	63	64	0.4209 / 0.8215	-0.0338

Table 10: Current long-rollout promotion gates after the norm-only continuation. The same-stream gate repeats one held-out stream for 1088 windows; the adversarial stitch alternates scene boundaries for 512 windows. These are stress gates, not a replacement for broader open-set video evaluation.

Lane	Windows	Runtime resets	Loss	Cosine	Late-early loss
Stage1 Memory-ALiBi final, same-stream repeat	1088	1	0.3273	0.8646	-0.0009
Stage2 norms final, same-stream repeat	1088	1	0.2794	0.8851	-0.0009
Stage1 Memory-ALiBi final, adversarial stitch	512	1	0.2850	0.8807	-0.0229
Stage2 norms final, adversarial stitch	512	1	0.2573	0.8924	-0.0193

```

cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-ttt-long-rollout-cactus-repeat-reset16-cuda.toml \
  --model "$LONG" \
  --steps 136 --batch-size 1 --no-full-grid

cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-ttt-long-rollout-stitched-reset-4x-cuda.toml \
  --model "$LONG" \
  --steps 64 --batch-size 1 --no-full-grid

cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-ttt-long-rollout-adversarial-stitch-reset-4x-cuda.toml \
  --model "$LONG" \
  --steps 64 --batch-size 1 --no-full-grid

Strict in-place conformance smoke:

cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- train-ttt \
  --config configs/production/vjepa21-ttt-stage1-stream-tbptt-inplace-mlp-strict-thirds-cuda.toml

Dense carried eval:

MODEL=target/burn-jepa-production-final-256/stage1-stream-tbptt-sigreg/ttt-model.mpk
cargo run --no-default-features --features cuda,sparse-patchify-cuda \
  --bin burn-jepa -- eval-ttt \
  --config configs/production/vjepa21-ttt-long-rollout-dense-eval-cuda.toml \
  --model "$MODEL" \
  --steps 16

```

Table 11: Carried-state coherence interventions for the selected sparse checkpoint. Positive loss gaps mean the persistent state is better.

Intervention	Loss / cosine	Coherence gap vs. free-run loss
Free-run persistent state	0.3334 / 0.8629	0.0000
Reset each frame	0.4639 / 0.8047	+0.1305
Reset each tubelet	0.4577 / 0.8074	+0.1243
Freeze fast updates	0.4639 / 0.8047	+0.1305
Shuffle order	0.4503 / 0.8106	+0.1169
Reverse order	0.5894 / 0.7520	+0.2560

Table 12: Representative 224px mask ablation at matched 20% context density from earlier controlled runs. TTT improves all listed policies in that run, but AutoGaze and patch-diff are close in quality.

Mask policy	Single loss	TTT loss	Single cosine	TTT cosine
Full-frame holdout	0.1925	0.1729	0.9190	0.9274
Random sparse	0.2181	0.1960	0.9078	0.9173
Patch-diff sparse	0.1962	0.1798	0.9163	0.9235
AutoGaze-style sparse	0.1970	0.1803	0.9165	0.9238

10 Final Inference Performance Appendix

Table 16 reports native WGPU inference telemetry for the deployed Bevy-style low-resolution feature pipeline. The run used an NVIDIA RTX PRO 6000 Blackwell Workstation Edition with driver 595.58.03. Each row loads the sharded V-JEPA 2.1 Burn package, runs four warmup frames, then measures 48 synchronized low-resolution inference frames while sampling `nvidia-smi` every 50 ms. AnyUp and Bevy display-panel conversion are disabled in this table so the measurement focuses on JEPA encode, sparse cache update, and low-resolution PCA. The sparse rows use a deterministic patch-diff stream with 30% cache-write density; the production bucketed-context policy widens that to a 50% encoder-token width to avoid shape churn. Sparse bucket prewarm is excluded from the timed frames.

These measurements clarify two deployment caveats. First, TTT adds modest latency and memory at 256px and larger latency at 512px, as expected from the adapter and carried-state work. Second, WGPU sparse patchify is not a guaranteed latency win at the default 30% write / 50% encode bucket. At this density, dense kernels remain highly competitive and sometimes faster; the sparse WGPU path mainly reduces 512px power in these rows. The larger latency wins reported in Table 13 still correspond to the CUDA sparse-patchify lane and lower effective token densities. The implementation now records cache-write width and encoder width separately so future sparse benchmarks cannot hide bucket padding behind a single sparsity number.

The reproducible command for this appendix is:

```
cargo build --release -p bevy_jepa --example gpu_inference_case \
  --no-default-features --features sparse-patchify-wgpu

target/release/examples/gpu_inference_case \
  --model base --mode sparse --resolution 512 \
  --frames 48 --warmup 4 --sample-ms 50 \
  --output-dir target/burn-jepa-gpu-inference-stage-matrix-20260517
```

11 Conclusion

SC-TTT is currently best framed as a reproducible sparse V-JEPA 2.1 systems method with a concrete algorithmic recipe and honest failure cases. The Burn model is numerically aligned with the official V-JEPA

Table 13: Representative AutoGaze-to-sparse-JEPA rolling throughput.

Lane	Resolution/density	Context tokens	Rolling ms	Rolling windows/s
ndarray CPU	224px / 1%	4	12.216	163.72
WebGPU-family	224px / 1%	4	33.824	59.13
CUDA	224px / 1%	4	3.918	510.41
CUDA	224px / 5%	20	6.769	295.46
CUDA	720p / 1%	72	3.937	507.99
CUDA	720p / 25%	1800	22.112	90.45

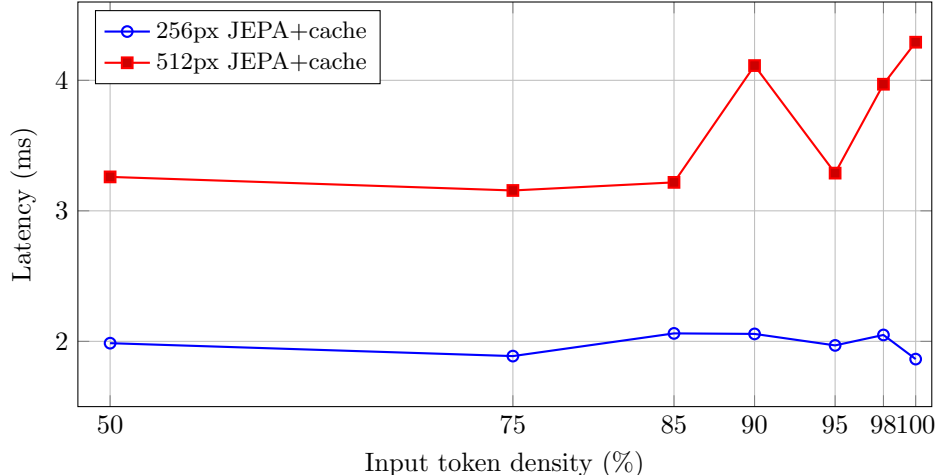


Figure 1: WGPU sparsity-latency sweep for the Bevy-style tiny JEPA+feature-cache path. The 100% point is the dense ordered baseline.

2.1 module, sparse CUDA inference is fast, and the verified checkpoint uses carried state more effectively than both base sparse V-JEPA and reset-state baselines. The LeJEPA-style regularizer is implemented and safe in this setting, but current evidence shows it as a neutral stabilizer rather than the reason SC-TTT works. The stress gates make the long-run claim more precise: simple unbounded carry was unstable, while Memory-ALiBi plus the norm-only continuation is stable on the measured 1088-window same-stream and 512-window adversarial-stitch gates. The strict In-Place TTT lane now has causal/update conformance checks and a real-checkpoint smoke run, but it remains an ablation rather than the shipped model. The evidence is not yet sufficient for theoretical originality, state-of-the-art claims, broad robustness claims, or unlimited-memory arbitrary-length rollout claims. The next step is larger real-mask held-out video, repeated statistics, external and downstream baselines, exact In-Place TTT reference parity, chunked dense diagnostics, and lower-overhead TTT backward kernels.

References

- [1] Lorenzo Mur-Labadia, Matthew Muckley, Amir Bar, Mido Assran, Koustuv Sinha, Mike Rabbat, Yann LeCun, Nicolas Ballas, and Adrien Bardes. V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning. *arXiv:2603.14482*, 2026. <https://arxiv.org/abs/2603.14482>.
- [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, and others. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. *arXiv:2506.09985*, 2025. <https://arxiv.org/abs/2506.09985>.
- [3] Randall Balestriero and Yann LeCun. LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics. *arXiv:2511.08544*, 2025. <https://arxiv.org/abs/2511.08544>.

Table 14: TTT forward+backward training-step sparsity sweep, batch 4.

Lane	10% sparse	100% dense	Gain
WebGPU-family dense-patch path	15.504 ms	22.696 ms	31.7%
WGPU flex-gmm pixel-skip path	8.790 ms	13.957 ms	37.0%
CUDA dense-patch path	14.627 ms	20.809 ms	29.7%
CUDA flex-gmm pixel-skip path	15.085 ms	19.835 ms	23.9%

Table 15: Minimum evidence table for contribution wording.

Claim type	Required evidence	Present?	Notes
Theoretical originality	New definition	partial	Definition 1 names a useful coherence metric, but it is evaluative rather than a central new theory.
Theoretical originality	Theorem/proposition	partial	Proposition 1 is a correctness guard for zero-init adapters, not a deep theory result.
Theoretical originality	Checkable proof	yes	Included for Proposition 1.
Theoretical originality	Assumptions	yes	Listed in Section 2.
Theoretical originality	Minimal implementation	yes	SC-TTT maps directly to the encoder adapter path.
Theoretical originality	Benchmark where method can lose	yes	Low-LR continuation, strict in-place smoke, and dense diagnostic failure.
Theoretical originality	Failure case	yes	Section 8.
Novel algorithm	Baseline	yes	Base sparse V-JEPA 2.1, dense/sparse image-to-video, reset, freeze, reverse/shuffle, dense, low-LR continuation, zero/no-op insertion.
Novel algorithm	Measurable improvement	yes	SC-TTT sparse beats base sparse V-JEPA by 0.1064 loss on the full manifest and improves the stage2 1088-window gate over stage1 by 0.0479 loss.
Novel algorithm	Ablation	yes	State reset, freeze, order, mask, density, backend, continuation, latent regularizer, in-place vs adapter.
Novel algorithm	Reason it works	partial	Memory-intervention gaps support the mechanism; more external validation is needed.
Novel algorithm	Case where it does not work	yes	Low-LR continuation, dense diagnostic allocator pressure, simple unbounded state carry, unreset scene stitching, and strict in-place not yet winning.
Novel algorithm	Short memorable name	yes	Sparse Carried-State TTT (SC-TTT).

- [4] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. *arXiv:2603.19312*, 2026. <https://arxiv.org/abs/2603.19312>.
- [5] Habibullah Akbar. Weak-SIGReg: Covariance Regularization for Stable Deep Learning. *arXiv:2603.05924*, 2026. <https://arxiv.org/abs/2603.05924>.
- [6] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. *arXiv:1909.13231*, 2019. <https://arxiv.org/abs/1909.13231>.
- [7] Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. In-Place Test-Time Training. *arXiv:2604.06169*, 2026. <https://arxiv.org/abs/2604.06169>.

Table 16: Final WGPU inference GPU-utilization appendix. VRAM is peak absolute device memory reported by NVML during the measured window; power and utilization are mean NVML samples.

Model	Mode	Res.	Write/enc.	Mean ms	FPS	GPU util.	VRAM MiB	Power W
Base	Dense	256	100/100%	18.87	52.86	60.8%	1806	80.9
Base	Dense	512	100/100%	28.48	34.93	57.2%	3702	144.3
Base	Sparse	256	30/50%	20.40	48.91	68.0%	1958	80.0
Base	Sparse	512	30/50%	31.11	31.99	54.7%	3906	120.7
SC-TTT	Dense	256	100/100%	21.32	46.81	56.7%	2245	83.7
SC-TTT	Dense	512	100/100%	33.86	29.41	66.0%	3618	153.8
SC-TTT	Sparse	256	30/50%	21.63	46.15	56.2%	2247	88.3
SC-TTT	Sparse	512	30/50%	33.54	29.68	58.7%	3913	129.0